Retail Data Platform

on Azure + Databricks

Project Design Document

v1.1 — adds ReSA canonical model

Author: Satyaki Guha

Status: Living document — open for edits

Last updated: April 2026

One-Line Pitch

End-to-end retail data platform on Azure + Databricks: streaming POS ingestion, ReSA-style sales audit, medallion lakehouse, and LLM-powered insights over 2+ years of multi-channel retail data.

Table of Contents

1. Project Vision & Goals

1.1 Why this project

The goal is a portfolio-grade retail data platform that showcases the skills relevant to modern data engineering roles — particularly in retail-adjacent companies — while leveraging the author's 7+ years of Oracle Retail QA experience as a differentiator.

1.2 Primary goals

- Demonstrate end-to-end data engineering across batch + streaming
- Showcase medallion lakehouse design with Delta Lake on Databricks
- Build a ReSA-style Sales Audit layer — the signature differentiator
- Integrate LLMs into a production-grade pipeline (not just a notebook demo)
- Flex Azure (DP-700) + Databricks (Associate + Professional) certifications
- Practice modern DevOps — Git, CI/CD, Infrastructure as Code

1.3 Non-goals

- Training custom ML/LLM models from scratch
- Building a production-grade UI or customer-facing app
- Handling truly massive datasets (we aim for ~100s of millions of rows, not billions)

2. Architecture Overview

2.1 High-level architecture

The diagram below shows the end-to-end flow. Three source channels (in-store POS, e-commerce, marketplace) feed through Azure Data Factory and Auto Loader into a governed Databricks lakehouse. Silver conforms all three channels into the Oracle ReSA canonical data model (SA_TRAN_HEAD / SA_TRAN_ITEM / SA_TRAN_DISC / SA_TRAN_TENDER / SA_TRAN_TAX / SA_TRAN_IGTAX / SA_STORE_DAY / SA_MISSING_TRAN), which then feeds both a Kimball analytics star schema and a ReSA-style audit engine. The LLM layer overlays the Gold and Audit layers; Synapse Serverless + Power BI provide the serving tier.

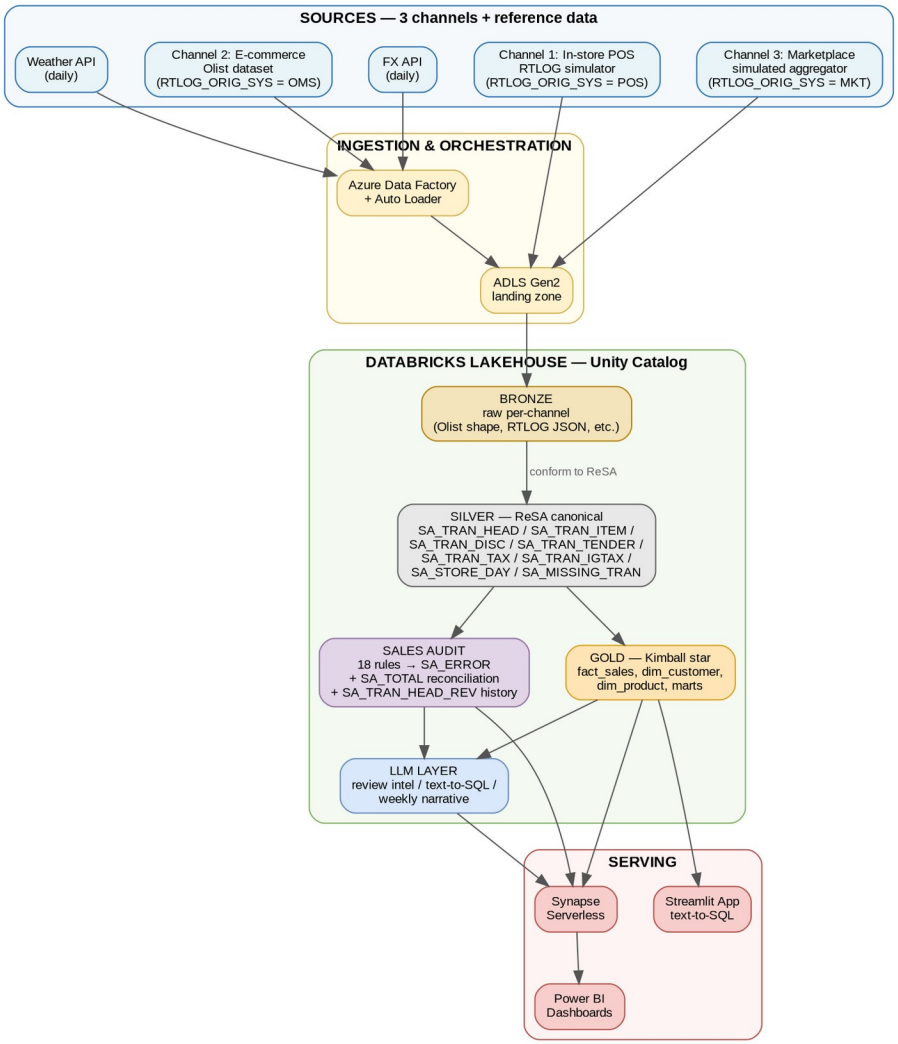

Figure 1 — End-to-end architecture

2.2 Design principles

- **Governed by Unity Catalog —** one source of truth for schemas, permissions, lineage
- **Incremental everywhere —** Auto Loader for files, CDC-style merges in Silver, materialized gold
- **Quarantine, don't drop —** bad records go to _quarantine tables with reason codes
- **Separation of concerns —** Sales Audit is its own schema, not mixed into Gold
- **LLM is a consumer, not a hard dependency —** the pipeline works fully without it

3. Technology Stack

The table below maps each architectural layer to the chosen technology, with a short rationale. Every choice is either Azure-native (for DP-700 alignment) or Databricks-native (to leverage the associate/professional certifications).

Layer	Technology	Why
Cloud	Azure (pay-as-you-go)	DP-700 alignment, industry standard
Storage	ADLS Gen2	Hierarchical namespace, Databricks-native
Compute	Databricks on Azure	Associate + Professional certs alignment
Governance	Unity Catalog	Standard for 2025+ Databricks deployments
Streaming ingestion	Auto Loader	Incremental file processing with checkpointing
Transformations	Delta Live Tables + PySpark	Declarative pipelines with expectations
Orchestration	ADF + Databricks Workflows	ADF for external hops, DBX Workflows inside
Warehousing	Databricks SQL + Synapse Serverless	Cost-effective Power BI source
BI	Power BI	Mainstream, Microsoft ecosystem
LLM	Databricks Foundation Model APIs	Zero extra accounts, unified billing
Vector store	Databricks Vector Search	Native integration with Delta
Secrets	Azure Key Vault	Industry standard
IaC	Terraform	Most portable skill
CI/CD	GitHub Actions + DAB	2025 recommended pattern
Testing	pytest + chispa	DataFrame unit testing
Observability	Azure Monitor + Log Analytics	Native Azure stack

4. Module Breakdown

Each module is designed to be independently demo-able. You should have something working at the end of every module, rather than a big-bang at the end.

4.1 Module 1 — POS RTLOG Simulator

Purpose: generate realistic retail transaction data that maps directly into the Oracle ReSA canonical tables (SA_TRAN_HEAD, SA_TRAN_ITEM, SA_TRAN_DISC, SA_TRAN_TENDER, SA_TRAN_TAX, SA_TRAN_IGTAX), so downstream modules consume real-shaped data from day one.

Why it matters: this is where the author's domain knowledge from Finance Foundations (UK retailer) and FS6/FS7 (Middle East retailer) shines brightest. Using the actual Oracle RMS 16.0 schema — not an invented one — makes the project instantly recognizable to any retail ETL hiring manager.

What it does

- Python service running as a scheduled job or Azure Container Instance
- Continuously writes JSON RTLOG records to an ADLS landing zone, partitioned by store_id/date/hour
- Simulates ~50 stores × 2 years of history = ~365M transaction lines (adjustable)
- Every record carries RTLOG_ORIG_SYS = 'POS' (channel discriminator in the ReSA model)
- Models realistic patterns: lunch rush, weekend peaks, Black Friday, seasonal trends
- Covers all major TRAN_TYPE values: SALE, RETURN, PVOID, PAIDIN, PAIDOUT, NOSALE, OPEN, CLOSE
- Injects ~2% "bad" records (missing tender, mismatched totals, duplicate TRAN_NO, off-hours voids) to exercise the Sales Audit layer

Record shape — RTLOG JSON (maps 1:1 to ReSA tables)

Each RTLOG record bundles one transaction with its child rows. The conformance job in Silver fans this out into the six SA_TRAN_* tables.

```
{
  "rtlog_orig_sys": "POS",
  "tran_head": {
    "store": 45,
    "business_date": "2024-03-15",
    "tran_seq_no": "STR045-TILL02-20240315-001234",
    "tran_datetime": "2024-03-15T12:34:22+05:30",
    "register": "TILL02",
    "tran_no": 1234,
    "cashier": "CASH1089",
    "salesperson": "SP0456",
    "tran_type": "SALE",
    "sub_tran_type": null,
    "status": "IMPORTED",
    "value": 594.82,
    "pos_tran_ind": "Y",
    "banner_no": 1,
```

```
    "ref_no1": "LOYALTY-9988123",
    "ref_no2": null
  },
  "tran_item": [
    {
      "item_seq_no": 1, "item_status": "ACTIVE", "item_type": "ITM",
      "item": "SKU-12345", "dept": 21, "class": 3, "subclass": 1,
      "qty": 2.0, "unit_retail": 299.00, "selling_uom": "EA",
      "tax_ind": "Y", "unit_retail_vat_incl": "Y",
      "total_igtax_amt": 26.82
    }
  ],
  "tran_disc": [
    {
      "item_seq_no": 1, "discount_seq_no": 1, "rms_promo_type": "PROMO",
      "promotion": 500123, "disc_type": "PROMO",
      "qty": 2.0, "unit_discount_amt": 15.00
    }
  ],
  "tran_tender": [
    {
      "tender_seq_no": 1, "tender_type_group": "CARD", "tender_type_id": 2,
      "tender_amt": 594.82, "cc_no": "************4521",
      "cc_auth_no": "A12345", "cc_entry_mode": "CHIP",
      "orig_currency": "INR", "orig_curr_amt": 594.82
    }
  ],
  "tran_tax": [],
  "tran_igtax": [
    {
      "item_seq_no": 1, "igtax_seq_no": 1, "tax_authority": "CGST",
      "igtax_code": "GST9", "igtax_rate": 0.09, "total_igtax_amt": 13.41
    },
    {
      "item_seq_no": 1, "igtax_seq_no": 2, "tax_authority": "SGST",
      "igtax_code": "GST9", "igtax_rate": 0.09, "total_igtax_amt": 13.41
    }
  ]
}
```


Key design decisions

- Column names and nullability match the Oracle RMS 16.0 data model exactly (see Appendix C)
- Credit card numbers are never stored in clear — only masked last-4, matching real PCI practice
- Uses inclusive tax (SA_TRAN_IGTAX) for Indian GST, not additive (SA_TRAN_TAX); simulator supports both modes via a config flag
- Unresolvable TRAN_NO gaps per register are reported into SA_MISSING_TRAN by the conformance job
- Configurable volume knob so you can scale down if costs pinch

Deliverables

- pos_simulator/ Python package with unit tests
- Dockerfile + deployment script
- Sample output files checked into repo (1K records for tests, not 365M)
- Conformance job that parses RTLOG JSON into the six child tables

4.2 Module 2 — Batch Sources & Additional Channels

Purpose: load the one-shot and periodically-refreshed reference data, plus provide the non-POS transaction channels (e-commerce and marketplace) that also get conformed to the ReSA canonical model in Silver.

Transaction sources (non-POS channels)

- **Olist Brazilian E-commerce (Channel 2) —** 9 tables loaded once via ADF into Bronze. Conformed to ReSA in Silver with RTLOG_ORIG_SYS = 'OMS'. CUST_ORDER_NO in SA_TRAN_ITEM provides the e-commerce linkage.
- **Marketplace aggregator (Channel 3) —** small simulated JSON feed mimicking a third-party aggregator (Uber Eats / Deliveroo style). Conformed to ReSA with RTLOG_ORIG_SYS = 'MKT'. Uses DROP_SHIP_IND = 'Y' on line items.

Reference data sources

- **FX rates (BRL/USD/EUR/INR) —** pulled daily from a free API (e.g., exchangerate.host). Used for SA_TRAN_TENDER.ORIG_CURRENCY conversion.
- **Weather data —** daily by city for correlation analysis (e.g., Open-Meteo)

Key design decisions

- ADF is used for these external hops, not Databricks, to demonstrate the full Azure stack
- FX and weather use ADF's REST copy activity with parameterized dates
- All sources land in ADLS Gen2 in their native shape before Databricks touches them
- Conformance happens in Silver, not Bronze — Bronze retains source-of-truth shape

Deliverables

- ADF pipelines exported as ARM templates (version-controlled)
- One Databricks notebook per source to load Bronze
- Three conformance notebooks in Silver: pos_to_resa, olist_to_resa, mkt_to_resa

4.3 Module 3 — Medallion Lakehouse

Purpose: the heart of the platform — clean, conformed, analytics-ready data, with Silver adopting the Oracle ReSA canonical model as the single shape all channels conform to.

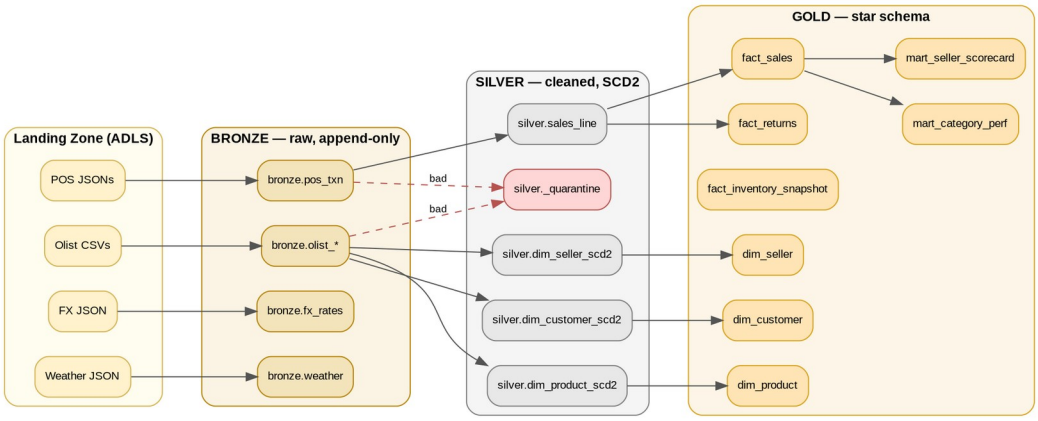

Figure 2 — Medallion layers (generic view)

Bronze layer

- Append-only, partitioned by ingestion date
- Each source retained in native shape (Olist stays Olist, RTLOG stays RTLOG)
- Schema evolution enabled via Auto Loader
- Retention: keep everything (cheap, enables re-processing)

Silver layer — ReSA canonical

Silver is NOT a generic cleaned schema — it is the Oracle ReSA canonical data model. All three channels are conformed into the same 8 transaction tables, differentiated only by RTLOG_ORIG_SYS. This is the single most important architectural decision in the project and directly mirrors how real Oracle Retail deployments work.

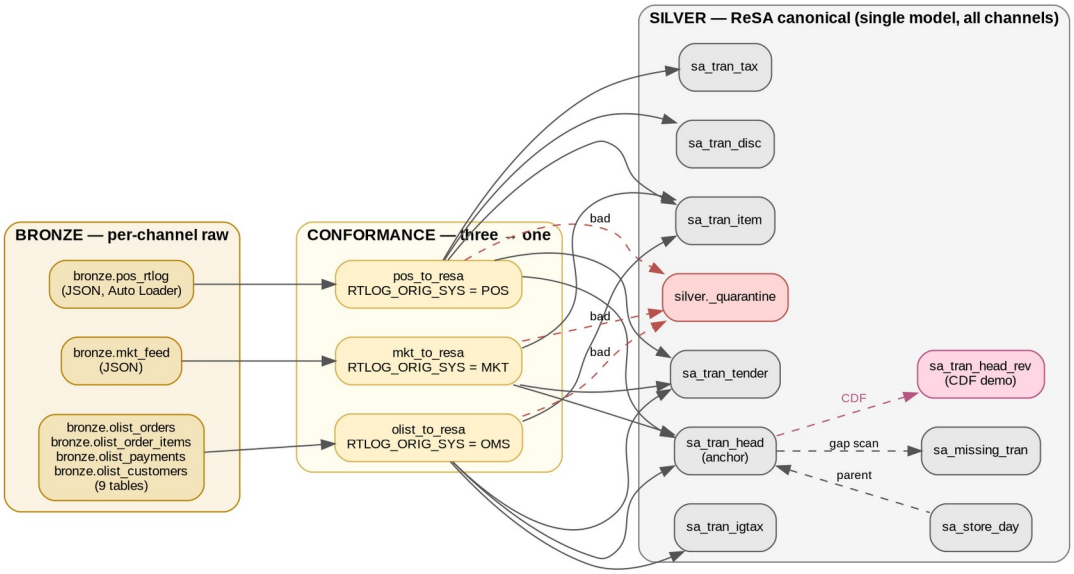

Figure 3 — Three channels conformed into ReSA canonical Silver

Silver table inventory (see Appendix C for full column specs)

- **sa_store_day —** parent row per store + business-date
- **sa_tran_head —** transaction anchor, 39 columns, PK (STORE, DAY, TRAN_SEQ_NO)
- **sa_tran_item —** line items, 51 columns
- **sa_tran_disc —** discounts per line (promo / coupon / manual)
- **sa_tran_tender —** tender payments (cash / card / voucher)
- **sa_tran_tax —** additive tax (sales tax / GST add-on)
- **sa_tran_igtax —** inclusive tax (VAT-style, price-inclusive GST)
- **sa_missing_tran —** gap detection for unaccounted TRAN_NOs per register
- **sa_tran_head_rev —** revision history of sa_tran_head (CDF demo)

Silver design decisions

- Column names and types match Oracle RMS 16.0 exactly — no drift
- Partitioning: STORE + DAY on all transaction tables, matching Oracle
- Y/N flag columns preserved (not booleans) for fidelity
- Bad records routed to silver._quarantine with rejection_reason
- DLT expectations enforce: referential integrity to sa_store_day, valid TRAN_TYPE, tender-sum = tran_head.VALUE

Revision history demo (sa_tran_head_rev)

To demonstrate the ReSA audit-trail pattern without building every _REV table, we implement it on SA_TRAN_HEAD only. Approach: enable Delta Change Data Feed (CDF) on silver.sa_tran_head; a small

streaming job reads the CDF and appends snapshots into silver.sa_tran_head_rev with an incremented REV_NO. This showcases Delta CDF, a type-2-style audit trail without a custom MERGE pipeline, and supports 'who changed what, when' queries.

Gold layer

- Classic Kimball star schema, built FROM the ReSA Silver tables
- fact_sales derives from sa_tran_head + sa_tran_item joined with dims
- Z-ordered on high-cardinality hot predicates (customer_id, order_date)
- Materialized marts for common dashboards (pre-aggregated for dashboard speed)

Deliverables

- DLT pipeline definitions in /transformations/dlt/
- Three conformance pipelines: pos_to_resa, olist_to_resa, mkt_to_resa
- sa_tran_head_rev CDF streaming job
- Data dictionary (docs/data_dictionary.md) referencing Appendix C
- ERD diagrams: ReSA Silver cluster + Gold star schema

4.4 Module 4 — Sales Audit Layer (ReSA-style)

Purpose: the signature piece — a reconciliation and audit layer modeled on Oracle ReSA, implementing the kind of logic the author has validated for years at Capgemini.

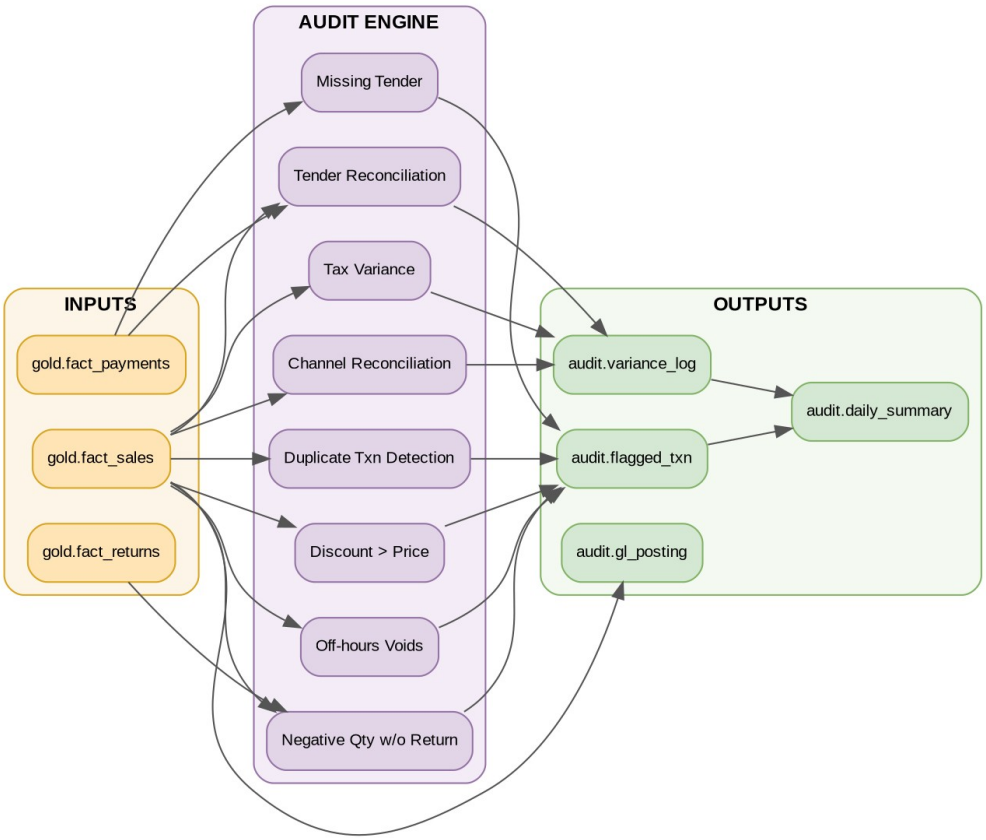

Figure 4 — Sales Audit rule engine

Audit rules catalog (target: 18 rules)

Each rule runs against the ReSA canonical Silver tables and, on failure, inserts a row into audit.sa_error with a distinct ERROR_CODE. ERROR_CODE values are seeded into audit.sa_error_codes on setup. This mirrors exactly how real ReSA handles rule failures — the ERROR_CODE becomes the join key between raw failures and their human-readable description.

#	Rule	Severity	ERROR_CODE	Trigger
1	Tender total = tran_head.VALUE	Critical	TENDER_VAR_GT_01	|Σ(tender_amt) − value| > 0.01
2	Channel daily total matches count	Critical	CHANNEL_COUNT_VAR	POS count vs SA_TRAN_HEAD count per RTLOG_ORIG_SYS
3	Duplicate TRAN_NO per	High	TRAN_NO_DUP	Same (store, register, tran_no)

#	Rule	Severity	ERROR_CODE	Trigger
	register			twice
4	Discount exceeds line price	High	DISC_GT_PRICE	Σ(tran_disc.unit_disc × qty) > tran_item.unit_retail × qty
5	Negative qty without return	High	NEG_QTY_NO_REF	tran_item.qty < 0 AND tran_type ≠ RETURN
6	Voids outside store hours	Medium	VOID_OOH	tran_type=PVOID outside 06:00-23:00
7	Missing tender record	Critical	MISSING_TENDER	tran_head with zero SA_TRAN_TENDER rows
8	Tax variance vs. expected rate	Medium	TAX_RATE_VAR	|tax_amt − qty×unit_retail×expected_rate| > 0.01
9	Unmatched TRAN_NO gap	High	TRAN_NO_GAP	Populates SA_MISSING_TRAN
10	Cash drawer over/short	Medium	CASH_OVER_SHORT	SUM(cash tenders) vs declared cash
11	Credit card unmasked	Critical	CC_NOT_MASKED	CC_NO doesn't start with mask chars
12	Invalid TRAN_TYPE	High	BAD_TRAN_TYPE	TRAN_TYPE not in code_type TRAT
13	Orphan tran_item	Critical	ITEM_NO_PARENT	tran_item without matching tran_head
14	Item price out of tolerance	Medium	PRICE_TOLERANCE	unit_retail deviates >20% from dim_product
15	Store closed but tran present	High	TRAN_STORE_CLOSED	tran outside sa_store_day window
16	IGTAX + TAX both present	Medium	DOUBLE_TAX	Both inclusive and additive tax on same line
17	Loyalty ID format invalid	Low	LOYALTY_FMT	REF_NO1 fails regex
18	Rounding variance too large	Medium	ROUND_VAR_GT_05	|ROUNDED_OFF_AMT| > 0.05

How rules populate SA_ERROR

When rule N fires on a transaction, a row is inserted into audit.sa_error with: STORE, DAY, ERROR_SEQ_NO (generated), TRAN_SEQ_NO (the offending tran), ERROR_CODE (from the table above), KEY_VALUE_1 (item_seq_no if applicable), and REC_TYPE='SART'. In parallel, the ERROR_IND column on the source table (sa_tran_head, sa_tran_item, etc.) is flipped to 'Y'. This exactly mirrors the production ReSA pattern.

GL posting table

- Daily aggregated debits/credits per account
- Accounts: Sales, Tax Payable, COGS, Discount Expense, Inventory, Cash-in-Bank
- Exported as a file to a mock "SAP landing zone" to echo the real integration pattern

Deliverables

- DLT pipeline per audit rule, each writing to audit.sa_error
- audit.sa_error and audit.sa_error_codes Delta tables (seeded from the 18 rules above)
- audit.sa_total_head + SA_TOTAL reconciliation logic for channel-level daily balancing
- audit.gl_posting table exported to mock SAP landing zone
- Power BI variance dashboard reading sa_error joined to sa_error_codes
- **Blog post:** "Building a ReSA-style Sales Audit Layer on Databricks" — this is marketing gold

4.5 Module 5 — LLM Intelligence Layer

Purpose: integrate LLMs into the pipeline for genuinely useful analytics, demonstrating that we can build around LLMs — not just wrap them in a chatbot.

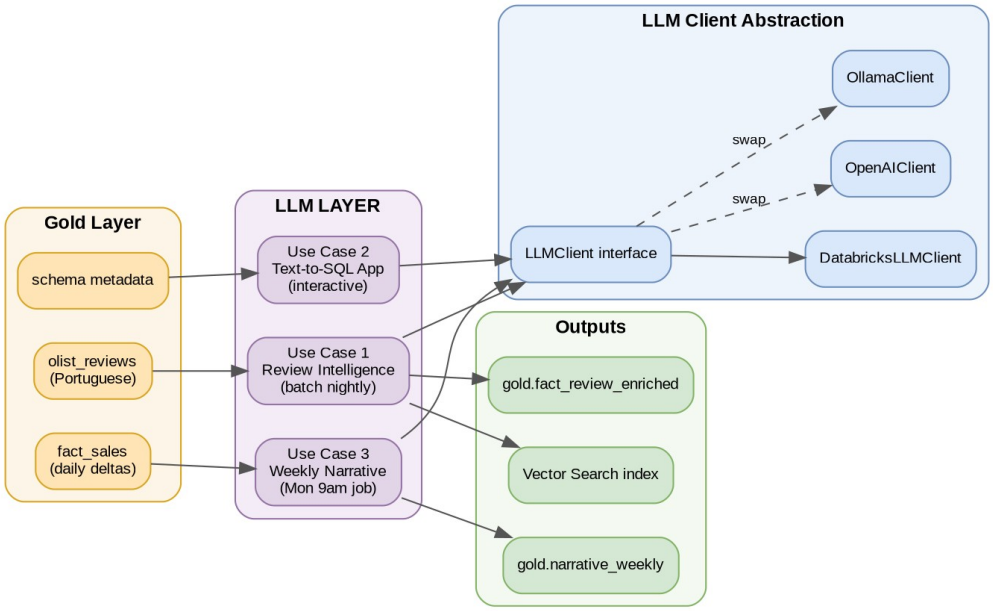

Figure 5 — LLM layer with provider abstraction

Use Case 1: Review Intelligence (batch)

- Nightly DLT job enriches new Portuguese reviews
- Each review → translated English, sentiment score, up to 3 theme tags from fixed taxonomy
- Taxonomy: delivery_speed, product_quality, packaging, price_value, seller_communication, accuracy_of_description
- Embeddings stored in Delta, indexed with Databricks Vector Search
- Feeds mart_seller_scorecard with sentiment aggregates

Use Case 2: Text-to-SQL Assistant (interactive)

- Streamlit app (deployed as a Databricks App)
- User types natural-language question; LLM generates SQL against Gold schema
- Guardrails: SELECT-only, schema-whitelist, row-limit cap
- Few-shot examples for common patterns (top-N, time-series, grouping)
- Every query logged for demo replay

Use Case 3: Weekly Narrative (scheduled)

- Monday 9am Databricks job

- Computes W-o-W deltas across key KPIs
- Feeds to LLM with prompt template: "You are a retail analyst. Write a 200-word Monday briefing..."
- Output to gold.narrative_weekly and optional Teams/Slack webhook

LLM Client abstraction (day-one design)

```
class LLMClient(Protocol):
    def complete(self, prompt: str, **kwargs) -> str: ...
    def embed(self, text: str) -> list[float]: ...

# Swap via config — no code changes elsewhere
```

Starting provider: Databricks Foundation Model APIs (Llama 3.3 70B Instruct endpoint for completions, bge-large-en for embeddings). Swappable to OpenAI gpt-4o-mini if cost or rate limits become an issue.

Deliverables

- /llm/clients/ with all three client implementations
- Streamlit app source + deployment config
- Sample weekly narratives checked in as illustration

4.6 Module 6 — Serving & Visualization

Purpose: make the platform usable by a business audience — the last mile.

Power BI dashboards (5 pages)

- **Executive KPIs —** revenue, AOV, order count, return rate, trend sparklines
- **Category Deep-dive —** revenue by category, top movers, margin view
- **Seller Scorecard —** sellers ranked by sales + sentiment themes from reviews
- **Sales Audit Variance —** daily variance trend, top-variance stores, flagged-txn drill-down
- **AI Briefing —** this week's narrative + text-to-SQL embed

Synapse Serverless sits in front of Gold as external tables — gives a "Fabric/Synapse" angle for DP-700 storytelling, and keeps Power BI queries cheap.

Deliverables

- .pbix file checked into repo
- Dashboard screenshots in README
- 3-minute demo video (critical for LinkedIn)

5. DevOps & Infrastructure as Code

5.1 Repository layout

```
retail-data-platform/
├── .github/workflows/        # GitHub Actions CI/CD
│   ├── pr-checks.yml         # lint + test on PR
│   └── deploy.yml            # DAB deploy on merge
├── infra/                    # Terraform
│   ├── main.tf
│   ├── modules/
│   └── environments/
│       ├── dev/
│       └── prod/
├── pos_simulator/            # Module 1
├── ingestion/                # Module 2 (ADF JSONs)
├── transformations/          # Module 3 (DLT pipelines)
│   ├── bronze/
│   ├── silver/
│   └── gold/
├── audit/                    # Module 4 (Sales Audit rules)
├── llm/                      # Module 5
│   ├── clients/
│   ├── review_enrichment/
│   ├── text_to_sql/
│   └── weekly_narrative/
├── serving/                  # Module 6 (Power BI)
├── tests/                    # pytest
├── databricks.yml            # DAB config
├── pyproject.toml
├── docs/
│   ├── architecture.md
│   ├── data_dictionary.md
│   └── runbook.md
└── README.md
```

5.2 CI/CD pipeline

Figure 6 — CI/CD pipeline from PR to prod deploy

5.3 Environment strategy

- **Two Unity Catalog catalogs:** dev and prod (single workspace is cheaper than two workspaces)
- **Terraform manages:** resource group, ADLS Gen2 account, Key Vault, Databricks workspace, ADF, Synapse Serverless pool
- **Secrets** (LLM API keys, SAS tokens) in Key Vault, referenced via Databricks secret scopes

- **Branch protection on main:** require PR + passing checks

6. Cost Management

6.1 Expected monthly cost (INR, pay-as-you-go)

Component	Estimate	Notes
Databricks DBUs	₹3,000-5,000	Job clusters only, spot instances, auto-terminate
Azure compute (ADF, containers)	₹500-1,000	Minimal usage
ADLS Gen2 storage	₹200-500	Few TB max
Synapse Serverless	₹200-500	Pay-per-query
LLM API calls	₹500-1,500	Depends on review volume
Key Vault, monitoring	₹100-200	Baseline
Total	₹4,500-8,700	Achievable with discipline

6.2 Cost controls

- **Budget alerts** at ₹5,000 and ₹10,000 on the resource group
- **Auto-terminate** all clusters after 10 min idle
- **Spot instances** for job clusters where possible
- **Scale-down knob** on the POS simulator (can drop from 365M to 50M if costs pinch)
- **Single resource group** for the entire project — nukeable in one command if pausing
- **Photon** enabled only where it pays back — not for tiny jobs

7. Build Timeline

Week	Focus	Demo-able output
1	Azure setup, Terraform, Unity Catalog, Olist into Bronze/Silver/Gold	Basic Power BI dashboard
2	Gold star schema, marts, Z-ordering, data dictionary	Complete batch pipeline
3	POS simulator build + local testing	Simulator emitting valid records
4	Auto Loader ingestion, DLT pipelines, expectations, quarantine	Streaming pipeline live
5	Sales Audit rules 1-9, variance dashboard	Audit layer functional
6	Sales Audit rules 10-18, GL posting, blog draft	Full audit layer complete
7	LLM client abstraction, review enrichment job	Seller scorecard with sentiment
8	Text-to-SQL app, weekly narrative job	All 3 LLM use cases live
9	Polish: README, demo video, Medium post, LinkedIn write-up	Portfolio-ready

*Each week boundary ends with **something that works**. At no point should the project be in a broken state for more than a day.*

8. Deliverables Checklist

Code & infrastructure

- ☐ GitHub repo (public) with complete source
- ☐ Terraform infra reproducible end-to-end
- ☐ GitHub Actions CI/CD green
- ☐ pytest suite with DataFrame tests
- ☐ Databricks Asset Bundle config

Content

- ☐ README with architecture diagram + 3-min demo video
- ☐ Data dictionary in docs/
- ☐ Runbook in docs/
- ☐ Blog post: ReSA-style Sales Audit on Databricks (Medium)
- ☐ LinkedIn post with architecture image
- ☐ 60-second teaser video (for LinkedIn feed)

Artifacts

- ☐ Power BI .pbix file
- ☐ ERD diagram
- ☐ Architecture diagram (SVG/PNG in repo)
- ☐ Sample weekly narratives

Appendix A — Decisions Log

Date	Decision	Rationale
Apr 2026	Use Azure + Databricks (not AWS)	DP-700 alignment, India market fit
Apr 2026	Include streaming (not batch-only)	Portfolio differentiation
Apr 2026	Include ReSA-style Sales Audit	Leverages author's domain expertise
Apr 2026	Databricks Foundation Model APIs as starting LLM	Unified billing, zero extra accounts
Apr 2026	Terraform for infra	Most portable IaC skill
Apr 2026	Olist dataset as base (not H&M or M5)	Best balance of realism, size, multi-entity
Apr 2026	Pay-out-of-pocket Azure	Full control, clean portfolio asset
Apr 2026 (v1.1)	Adopt Oracle ReSA canonical model in Silver	Biggest portfolio differentiator; matches author's day-job expertise; makes all 3 channels conform cleanly
Apr 2026 (v1.1)	Scope to 'ReSA Core 11' tables	Full ReSA is 50+ tables; 11 covers the transaction + audit story without bloat
Apr 2026 (v1.1)	Use RTLOG_ORIG_SYS as channel discriminator	This is the actual ReSA pattern; cleaner than a custom channel column
Apr 2026 (v1.1)	Implement _REV pattern on SA_TRAN_HEAD only	Demonstrates the audit-trail concept via Delta CDF without building every _REV table
Apr 2026 (v1.1)	Add simulated marketplace as 3rd channel	Makes the conformance story real — three channels → one canonical model

Appendix B — Open Questions

1. **Streaming source:** file-drop-based (simpler) or Event Hub (more impressive)? Current plan: file-drop for v1, optional Event Hub for v2.
2. **Audit rule count:** target 18 rules. Full list (10-18) to be catalogued in the Module 4 deep-dive.
3. **Text-to-SQL app hosting:** Databricks Apps (cleanest) or a separate Azure Container App (more portable)?
4. **Demo data privacy:** should customer_email in Olist be pseudonymized even though the dataset is public? Lean yes — shows privacy awareness.
5. **Retention policy:** how long do we keep Bronze? Proposal: 90 days for cost, forever if budget allows.
6. **Monitoring deep-dive:** do we build a Databricks SQL dashboard for pipeline health, or rely on Azure Monitor?

Appendix C — ReSA Canonical Model Reference

This appendix is the authoritative column spec for the 11 Silver-layer tables, extracted from the Oracle Retail Merchandising System (RMS) 16.0 data model. Our DLT / PySpark code must match these names, types, and nullability. The full extended reference is also maintained in docs/resa_reference.md in the repo.

Scope — the ReSA Core 11

#	Table	Role	Cols
1	SA_STORE_DAY	Parent — one row per store + business-date	13
2	SA_TRAN_HEAD	Transaction header — the anchor	39
3	SA_TRAN_ITEM	Line items	51
4	SA_TRAN_DISC	Discounts per line	22
5	SA_TRAN_TENDER	Tender / payment	29
6	SA_TRAN_TAX	Additive tax (sales tax / GST add-on)	11
7	SA_TRAN_IGTAX	Inclusive (VAT-style) tax	16
8	SA_MISSING_TRAN	Gap detection for unaccounted TRAN_NOs	6
9	SA_ERROR	Every audit-rule failure lands here	17
10	SA_ERROR_CODES	Lookup for error types (seeded with our 18)	12
11	SA_TOTAL_HEAD	Named-total definitions for reconciliation	31
demo	SA_TRAN_HEAD_REV	Revision history pattern (CDF-backed)	39+REV_NO

C.1 SA_TRAN_HEAD (anchor table)

One row per transaction. PK: (STORE, DAY, TRAN_SEQ_NO). Parent: SA_STORE_DAY via STORE_DAY_SEQ_NO. The 39 columns include the channel discriminator RTLOG_ORIG_SYS (default 'POS'), the workflow STATUS, and 7 REF_NO* flex fields.

#	Column	PK/M	Type	Notes
1	STORE	P/M	NUMERIC(10)	Partitioning
2	DAY	P/M	NUMERIC(3)	Partitioning (SA_DATE_HASH)
3	TRAN_SEQ_NO	P/M	NUMERIC(20)	Sales-audit tran ID

#	Column	PK/M	Type	Notes
4	REV_NO	M	NUMERIC(3)	Revision counter
5	STORE_DAY_SEQ_NO	FK/M	NUMERIC(20)	→ SA_STORE_DAY
6	TRAN_DATETIME	M	Date(7)	
7	REGISTER		VARCHAR(5)	Till ID
8	TRAN_NO		NUMERIC(10)	POS-assigned, unique per register
9	CASHIER		VARCHAR(10)	
10	SALESPERSON		VARCHAR(10)	
11	TRAN_TYPE	M	VARCHAR(6)	SALE/RETURN/PVOID/ PAIDIN/PAIDOUT/...
12	SUB_TRAN_TYPE		VARCHAR(6)	code_type TRAS
13-15	ORIG_TRAN_NO/TYPE/REG_NO		—	For post-void
16-19	REF_NO1...REF_NO4		VARCHAR(30)	Flex fields
20	REASON_CODE		VARCHAR(6)	
21-24	VENDOR_NO, INVC_NO, PAYMENT_REF_NO, PROOF_OF_DELIVERY_NO		—	PAIDIN/PAIDOUT support
25	STATUS	M	VARCHAR(6)	imported/validated/errored/ approved/posted
26	VALUE		NUMERIC(20,4)	Grand total
27	POS_TRAN_IND	M	VARCHAR(1)	Y/N
28-29	UPDATE_DATETIME, UPDATE_ID	M	—	Audit cols
30	ERROR_IND	M	VARCHAR(1)	Y if SA_ERROR row exists
31	BANNER_NO		NUMERIC(4)	Multi-banner retailers
32-33	ROUNDED_AMT, ROUNDED_OFF_AMT		NUMERIC(20,4)	Rounding rules
34	CREDIT_PROMOTION_ID		NUMERIC(10)	
35-37	REF_NO25...REF_NO27		VARCHAR(30)	Flex fields

#	Column	PK/M	Type	Notes
38	RTLOG_ORIG_SYS	M	VARCHAR(3)	'POS' | 'OMS' | 'MKT' — channel discriminator
39	TRAN_PROCESS_SYS		VARCHAR(3)	

C.2 SA_TRAN_ITEM (51 columns, abridged)

PK: (STORE, DAY, TRAN_SEQ_NO, ITEM_SEQ_NO). Child of SA_TRAN_HEAD. Notable columns beyond the obvious: CATCHWEIGHT_IND for variable-weight grocery, DROP_SHIP_IND for marketplace flag, CUST_ORDER_NO for e-commerce linkage, UNIT_RETAIL_VAT_INCL for inclusive-tax mode (default 'N').

#	Column	PK/M	Type	Notes
1-3	STORE, DAY, TRAN_SEQ_NO	P/M	—	Parent key
4	ITEM_SEQ_NO	P/M	NUMERIC(4)	Line number
5	ITEM_STATUS	M	VARCHAR(6)	ACTIVE/VOIDED/REFUNDED
6	ITEM_TYPE	M	VARCHAR(6)	ITM/GCN/REF/TND
7	ITEM		VARCHAR(25)	SKU
11-13	DEPT, CLASS, SUBCLASS		NUMERIC(4)	Merch hierarchy
14	QTY		NUMERIC(12,4)	
15	UNIT_RETAIL		NUMERIC(20,4)	Sale price
17-19	OVERRIDE_REASON, ORIG_UNIT_RETAIL, STANDARD_ORIG_UNIT_RETAIL		—	Price overrides
20	TAX_IND	M	VARCHAR(1)	
22	ERROR_IND	M	VARCHAR(1)	Y if SA_ERROR row exists
23	DROP_SHIP_IND	M	VARCHAR(1)	Marketplace flag
27	RETURN_REASON_CODE		VARCHAR(6)	
33-36	REF_NO5...REF_NO8		VARCHAR(30)	Flex fields
38	CATCHWEIGHT_IND		VARCHAR(1)	Variable-weight grocery
42	UNIT_RETAIL_VAT_INCL	M	VARCHAR(1)	Default 'N'
43	TOTAL_IGTAX_AMT		NUMERIC(20,4)	Line-level inclusive tax

#	Column	PK/M	Type	Notes
45	CUST_ORDER_NO		VARCHAR(48)	E-commerce linkage
46-47	CUST_ORDER_DATE, FULFILL_ORDER_NO		—	E-commerce linkage
...	(remaining 30 columns)			See docs/resa_reference.md

C.3 SA_TRAN_TENDER (29 columns)

PK: (STORE, DAY, TRAN_SEQ_NO, TENDER_SEQ_NO). Multi-tender per transaction (split tender). Multi-currency via ORIG_CURRENCY + ORIG_CURR_AMT.

#	Column	PK/M	Type	Notes
1-3	STORE, DAY, TRAN_SEQ_NO	P/M	—	Parent key
4	TENDER_SEQ_NO	P/M	NUMERIC(4)	
5	TENDER_TYPE_GROUP	M	VARCHAR(6)	CASH / CARD / CHECK / VOUCHER
6	TENDER_TYPE_ID	M	NUMERIC(6)	
7	TENDER_AMT	M	NUMERIC(20,4)	
8	CC_NO		VARCHAR(40)	Store masked only
10-15	CC_AUTH_NO, ENTRY_MODE, ...		—	Card details
16-18	VOUCHER_NO, COUPON_NO, COUPON_REF_NO		—	Non-card tenders
19-22	REF_NO9...REF_NO12		VARCHAR(30)	Flex
23	ERROR_IND	M	VARCHAR(1)	
24-25	CHECK_ACCT_NO, CHECK_NO		—	Cheque details
28	ORIG_CURRENCY		VARCHAR(3)	Multi-currency support
29	ORIG_CURR_AMT		NUMERIC(20,4)	Amount in original currency

C.4 SA_TRAN_DISC, SA_TRAN_TAX, SA_TRAN_IGTAX, SA_MISSING_TRAN, SA_STORE_DAY

Condensed listings — full column specs in docs/resa_reference.md in the repo.

SA_TRAN_DISC (22 cols)

PK: (STORE, DAY, TRAN_SEQ_NO, ITEM_SEQ_NO, DISCOUNT_SEQ_NO, RMS_PROMO_TYPE). Key cols: PROMOTION, DISC_TYPE (PROMO/MANUAL/COUPON/EMP), COUPON_NO, QTY, UNIT_DISCOUNT_AMT, PROMO_COMP.

SA_TRAN_TAX (11 cols) — additive tax

PK: (STORE, DAY, TRAN_SEQ_NO, TAX_SEQ_NO). Key cols: TAX_CODE, TAX_AMT, plus REF_NO17-20 flex.

SA_TRAN_IGTAX (16 cols) — inclusive/VAT tax

PK: (STORE, DAY, TRAN_SEQ_NO, ITEM_SEQ_NO, IGTAX_SEQ_NO). Per-item per-tax-authority. Key cols: TAX_AUTHORITY (e.g. CGST, SGST), IGTAX_CODE, IGTAX_RATE, TOTAL_IGTAX_AMT.

SA_MISSING_TRAN (6 cols)

Populated during import reconciliation. Cols: MISS_TRAN_SEQ_NO (PK), STORE_DAY_SEQ_NO (FK), REGISTER, TRAN_NO (the missing number), STATUS, RTLOG_ORIG_SYS.

SA_STORE_DAY (13 cols) — parent

Cols: STORE_DAY_SEQ_NO (PK), BUSINESS_DATE, STORE, DAY, INV_BUS_DATE_IND, INV_STORE_IND, STORE_STATUS, STORE_CLOSED_DATETIME, DATA_STATUS (P=partial, F=full, G=purging), AUDIT_STATUS, AUDIT_CHANGED_DATETIME, FILES_LOADED, OMS_FILES_LOADED.

C.5 SA_ERROR & SA_ERROR_CODES

SA_ERROR (17 cols) holds every rule failure. PK: (STORE, DAY, ERROR_SEQ_NO). Exactly one of BAL_GROUP_SEQ_NO / TOTAL_SEQ_NO / TRAN_SEQ_NO is populated per row (check constraint). REC_TYPE distinguishes: SART=tran error, SABG=balance group error, SATT=total error.

SA_ERROR_CODES (12 cols) is the lookup. We seed it once with our 18 rule codes (TENDER_VAR_GT_01, DISC_GT_PRICE, etc.) — each row has ERROR_CODE (PK), ERROR_DESC, REC_SOLUTION, STORE_OVERRIDE_IND, REQUIRED_IND.

C.6 SA_TRAN_HEAD_REV — revision history demo

All 39 columns of SA_TRAN_HEAD plus REV_NO as the 4th PK column. Implementation: enable Delta CDF on silver.sa_tran_head; a streaming job reads the CDF and appends snapshots here with incremented REV_NO. Supports 'who changed what, when' queries as a demonstrator of the full ReSA audit-trail pattern.

C.7 RTLOG_ORIG_SYS — channel discriminator

Value	Channel	Source in our project
POS	In-store point-of-sale	Module 1 POS RTLOG simulator
OMS	Order management (e-commerce)	Olist dataset, conformed in Silver
MKT	Marketplace aggregator	Simulated feed (small) for third channel

C.8 Code Types we seed

ReSA uses CODES / CODE_DETAIL tables for enumerations. Not part of our Core 11 but populated as seed data. Minimum we need:

code_type	Meaning	Values
TRAT	Transaction types	SALE, RETURN, PVOID, PAIDIN, PAIDOUT, NOSALE, OPEN, CLOSE
SASS	Store statuses	OPEN, CLOSED
SAAS	Audit statuses	UNAUDITED, INPROGRESS, AUDITED, POSTED
SASI	Item statuses	ACTIVE, VOIDED, REFUNDED
SART	Error record types	SART=tran, SABG=bal group, SATT=total

Source: Oracle Retail Merchandising System 16.0 Data Model (rms-160-dm-02.pdf, 2017). Extracted for educational / portfolio use.

— End of document — v1.1. Open for edits. —